GREYNOC CYBERSECURITY OPERATIONS

greynoc.com

PUBLIC RELEASE DOCUMENT

GreyNOC EDC Operator Guide

Android Defensive Operator Stack - v1.8

This guide equips defense operators with a no-root Android cyber workspace using Termux, PRoot Debian, and a curated defensive tool set.
Covers installation, tool syntax, field notes, OpSec, and authorized lab use.

Document ID	GN-OPS-EDCAND-001
Version	v1.8
Effective Date	2026-06-04
Document Owner	GreyNOC Cybersecurity Operations
Classification	PUBLIC / APPROVED FOR EXTERNAL RELEASE

Replace all placeholders. Run security and communications review before publication.

Document Control

Title	EDC Operator Guide — Android Defensive Stack
Document ID	GN-OPS-EDCAND-001
Classification	PUBLIC / APPROVED FOR EXTERNAL RELEASE
Owner	GreyNOC Cybersecurity Operations
Approvers	Owner · Security · Communications
Effective Date	2026-06-04

Revision History

Version	Date	Author	Summary of Change
v0.1	2026-05-01	GreyNOC	Initial draft from GitHub source.
v0.9	2026-05-15	GreyNOC	Review candidate — expanded tool syntax.
v1.0	2026-06-04	GreyNOC	Public release — template-aligned branding.
v1.1	2026-06-04	GreyNOC	QA pass: fixed orphan pages, table cell consistency, cover gap, code comment style.
v1.8	2026-06-04	GreyNOC	Connected cover red frame lines and bumped public release version.

Release Rules

This document is a public release guide. It contains only externally approved claims, sanitized procedures, and release-safe contact information. Exploit instructions, internal toolchain details, live IP addresses, customer environments, and unreleased capability descriptions are explicitly excluded.

IMPORTANT — Run security and communications review before publishing updated versions. Archive the source file, final PDF, approval notes, and checksum per GreyNOC release procedure.

Table of Contents

01 Scope and Responsible-Use Notice

This guide is written for public education, field readiness, and legitimate technical operations. It is intended for operators, students, SOC learners, technicians, home-lab builders, and small-business defenders who want a portable command-line environment on an Android device — without rooting the phone, replacing Android, or installing custom firmware.

Use every tool covered in this guide only on devices, networks, accounts, and lab environments you own or have **explicit written permission** to administer or test. Unauthorized scanning, exploitation, credential attacks, interception, or bypassing access controls may violate the Computer Fraud and Abuse Act (18 U.S.C. § 1030), Michigan MCL 752.791, organizational policy, and service terms.

> **WARNING —** Michigan operators: document written authorization before running any active tool. A screenshot of an email approval is not sufficient for professional engagements — use a signed scope-of-work or rules-of-engagement document.

Out of Scope

This guide does not provide exploit instructions, credential theft guidance, stealth or evasion techniques, malware, monitor-mode or packet-injection setup, custom ROM flashing, bootloader unlock procedures, or unauthorized access workflows.

02 What This Guide Installs

This stack requires no root, no bootloader unlock, and no custom ROM. Android stays installed and running as the host OS. The operator adds a terminal environment and a Debian Linux userland on top of the existing Android installation.

Component	Role	Root Required
Android (stock)	Host OS — remains untouched throughout	No
F-Droid	Open-source app catalog hosting maintained Termux builds	No
Termux	Terminal emulator and Linux package environment	No
PRoot Distro	Manages Linux userland installs inside Termux userspace	No
Debian (PRoot)	Full apt ecosystem — scripting, SSH, DNS, lab tools	No

TIP — PRoot runs a Linux filesystem entirely in userspace — no kernel modifications are made. Some low-level networking features (raw sockets, monitor mode, packet injection) are blocked by Android's kernel permissions. This is expected and by design for a no-root, no-flash setup.

03 Operator Requirements

Requirement	Recommendation
Android device	Phone or tablet you control at the app level — Android 8.0+
Storage	8 GB free minimum; 20 GB or more strongly recommended for lab use
Network	Stable Wi-Fi for initial package downloads (~500 MB total)
Power	Keep charged or plugged in during installation phases
Keyboard	Optional Bluetooth keyboard — strongly recommended for field productivity
App sources	F-Droid only for Termux — avoid random APK download sites
Backup	Back up personal phone data before beginning any installation

TIP — A compact Bluetooth keyboard (60% or 65% layout) dramatically improves usability for terminal work in the field. Pair it once and it reconnects automatically.

04 Installing F-Droid and Termux

Step 1 — Install F-Droid

Download the F-Droid APK from the official site at **f-droid.org**. F-Droid is a free, open-source app catalog. The actively maintained Termux build is distributed through F-Droid. The Google Play Store version of Termux is outdated and should not be used.

> **WARNING —** Enable 'Install from unknown sources' for your browser in Android Settings > Apps before downloading the F-Droid APK. After installing F-Droid you can re-restrict that setting.

Step 2 — Install Termux from F-Droid

Open F-Droid, search for Termux, and install it. Termux provides a complete Linux terminal environment on Android without requiring root access.

Step 3 — Grant Storage Access

```
termux-setup-storage
```

When Android prompts, tap **Allow**. This creates **~/storage/** symlinks so Termux can read and write the phone's Downloads, Pictures, and shared folders.

Step 4 — Update Termux Packages

```
pkg update && pkg upgrade -y
```

This updates the Termux package index and upgrades all installed packages. Run this command every time you start a new session to keep the environment current.

Step 5 — Install Required Termux Packages

```
pkg install proot-distro git wget curl nano openssh -y
```

Package	Role	Key Command
`proot-distro`	Linux distro manager for Termux	proot-distro install / login / remove
`git`	Version control	git clone
`wget`	HTTP file downloader	wget -O file.txt
`curl`	URL data transfer and API calls	curl -s https://api.example.com
`nano`	Simple terminal text editor	nano ~/file.txt
`openssh`	SSH client, keygen, scp, sftp	ssh user@host

05 Installing Debian Inside Termux

Step 6 — Install Debian

```
proot-distro install debian
```

Downloads a Debian Linux filesystem (~200–300 MB) into Termux. Installed path: **~/.local/share/proot-distro/installed-rootfs/debian/**

Step 7 — Log Into Debian

```
proot-distro login debian
```

You are now inside Debian as root. The prompt changes from **$** (Termux) to # (Debian root). All **apt** commands are available here.

Step 8 — Update Debian

```
apt update && apt upgrade -y
```

Step 9 — Install Everyday Linux Tools

```
apt install sudo git curl wget nano vim python3 python3-pip \

dnsutils whois netcat-openbsd openssh-client -y
```

Package	Purpose	Common Syntax
`sudo`	Run commands as root from user account	sudo
`git`	Source code version control	git clone
`curl`	Transfer data from URLs and APIs	curl -s https://api.example.com
`wget`	Download files from the web	wget -O file.txt
`nano`	Beginner-friendly terminal editor	nano ~/file.txt
`vim`	Advanced modal text editor	vim ~/file.txt (i=insert :wq=save)
`python3`	Scripting and automation runtime	python3 script.py
`python3-pip`	Python package installer	pip3 install requests
`dnsutils`	DNS query tools: dig, nslookup	dig example.com A
`whois`	Domain and IP registration lookup	whois example.com

Package	Purpose	Common Syntax
`netcat-openbsd`	TCP/UDP Swiss army knife	nc -zv 192.168.1.1 22
`openssh-client`	SSH remote shell and file transfer	ssh user@host

Step 10 — Create a Normal Operator User

```
adduser greyuser

usermod -aG sudo greyuser

exit

# Re-login as the new user:

proot-distro login debian --user greyuser
```

Replace **greyuser** with your preferred username. Working as a non-root user is best practice — it mirrors production Linux environments and limits accidental damage from typos.

TIP — To drop back to Debian root at any time: proot-distro login debian (no --user flag). Useful for fixing sudo configs or installing system-level packages.

06 GreyNOC EDC Workspace Structure

A clean folder structure keeps notes, scripts, logs, evidence, and client materials separated. Run this inside Debian as your operator user:

```
mkdir -p ~/greynoc-edc/{notes,scripts,logs,reports,clients,lab,tools}

mkdir -p ~/greynoc-edc/reports/{templates,evidence,completed}

touch ~/greynoc-edc/notes/field-notes.md

touch ~/greynoc-edc/logs/session-log.txt
```

Folder	Purpose
notes/	Operator notes, checklists, runbooks, reminders
scripts/	Helper scripts (bash, python)
logs/	Local session notes and authorized command output
reports/templates/	Blank report templates for reuse
reports/evidence/	Raw output and screenshots — sanitized copies only
reports/completed/	Finalized deliverables
clients/	Per-engagement folders — never store secrets in plain text
lab/	Home-lab and CTF artifacts
tools/	Custom utilities and helper files

TIP — Add a README.md to each client folder documenting the engagement scope and authorization reference. This protects you and keeps engagements auditable.

07 Quick Launch Command

Create a shell command so you can start your Debian workspace from Termux with a single short command. Run this in **Termux** (not inside Debian):

```
echo 'proot-distro login debian --user greyuser' > $PREFIX/bin/linux-phone

chmod +x $PREFIX/bin/linux-phone
```

To start the workspace:

```
linux-phone
```

TIP — Add an alias to ~/.bashrc in Termux for even faster access: alias lp='proot-distro login debian --user greyuser'

08 Baseline Toolset — Full Syntax Reference

Install the full baseline inside Debian as your operator user:

```
sudo apt update

sudo apt install -y \

git curl wget nano vim tree unzip zip \

python3 python3-pip python3-venv \

dnsutils whois netcat-openbsd \

openssh-client jq htop tmux
```

SSH — openssh-client

SSH is the primary remote access tool for any defense operator. Use it to administer authorized servers, proxy through bastion hosts, and transfer files securely. Always prefer **Ed25519** keys over RSA.

Command	Description
`ssh user@host`	Connect to a remote host on port 22
`ssh -p 2222 user@host`	Connect on a non-standard port
`ssh -i ~/.ssh/id_ed25519 user@host`	Specify a private key explicitly
`ssh -L 8080:localhost:80 user@host`	Local port forward — tunnel remote:80 to local:8080
`ssh -R 9090:localhost:22 user@host`	Reverse tunnel — expose local:22 via remote:9090
`ssh -J jump@bastion user@target`	ProxyJump through a bastion host
`ssh-keygen -t ed25519 -C 'op@greynoc'`	Generate an Ed25519 key pair
`ssh-copy-id user@host`	Install public key on a remote host
`scp file.txt user@host:~/`	Copy file to remote host
`scp user@host:~/file.txt .`	Copy file from remote host
`sftp user@host`	Interactive secure file transfer session

DNS Tools — dig, nslookup, host

DNS reconnaissance is a foundational defensive skill. These tools verify records, investigate suspicious domains, and troubleshoot resolution issues.

Command	Description
`dig example.com`	Query all default records
`dig example.com A`	Query IPv4 address records
`dig example.com MX`	Query mail exchange records
`dig example.com TXT`	Query TXT records — SPF, DKIM, verification tokens
`dig example.com NS`	Query nameserver records
`dig example.com AAAA`	Query IPv6 records
`dig @8.8.8.8 example.com`	Query using a specific resolver
`dig +short example.com`	Short output — answer section only
`dig -x 93.184.216.34`	Reverse DNS lookup (PTR record)
`nslookup example.com`	Interactive DNS lookup
`host example.com`	Simple forward and reverse lookup

TIP — dig +short is ideal for scripting. Combine with sort and awk: dig +short example.com MX | sort -n

WHOIS — Domain and IP Registration

Command	Description
`whois example.com`	Domain registration, registrar, nameservers, dates
`whois 93.184.216.34`	IP ownership and ASN information
`whois -h whois.arin.net 93.184.216.34`	Query ARIN directly for IP block info

Netcat — nc (netcat-openbsd)

Netcat is the Swiss army knife of TCP/UDP networking. Use it for port verification, banner grabbing, and simple data transfer on authorized systems.

Command	Description
`nc -zv 192.168.1.1 22`	TCP port check — verbose result
`nc -zv 192.168.1.1 20-25`	TCP port range check
`nc -zvu 192.168.1.1 53`	UDP port check on port 53

Command	Description
`nc 192.168.1.1 80`	Interactive TCP connection to port 80
`nc -l -p 9999`	Listen on port 9999 (receive mode)
`echo 'test' | nc 192.168.1.1 9999`	Send string to a listening host
`nc -w 3 192.168.1.1 443`	Set connection timeout to 3 seconds

WARNING — Always confirm written authorization before port-scanning any host.

tmux — Terminal Multiplexer

tmux keeps sessions alive when the phone screen sleeps or Termux loses focus. It lets you run multiple terminal panes and windows in one session — essential for field work.

Command / Shortcut	Description
`tmux`	Start a new session
`tmux new -s ops`	Start a named session
`tmux attach -t ops`	Reattach to a named session
`tmux ls`	List all active sessions
`tmux kill-session -t ops`	Kill a session
`Ctrl+b c`	Create a new window
`Ctrl+b n / p`	Next / previous window
`Ctrl+b %`	Split pane vertically
`Ctrl+b "`	Split pane horizontally
`Ctrl+b arrow keys`	Move between panes
`Ctrl+b d`	Detach (session stays running in background)
`Ctrl+b [`	Enter scroll/copy mode — q to exit

TIP — Start a tmux session immediately after logging into Debian. Name sessions by task: tmux new -s recon or tmux new -s ssh-work

jq — JSON Processor

jq parses, filters, and transforms JSON. Critical for working with API responses, log exports, and threat intelligence feeds.

Command	Description
`cat data.json | jq .`	Pretty-print JSON
`cat data.json | jq '.key'`	Extract a top-level key
`cat data.json | jq '.[] | .name'`	Extract 'name' from each array element
`cat data.json | jq 'length'`	Count elements in an array
`curl -s https://api/data | jq '.results[]'`	Chain curl and jq for live API parsing

Python3 — Scripting and Automation

Python is the core scripting language for GreyNOC operators — log parsing, API integrations, report generation, and custom defensive tooling.

Command	Description
`python3 --version`	Confirm Python version
`python3 script.py`	Run a Python script
`python3 -c 'print("test")'`	Run inline Python code
`pip3 install requests`	Install a Python package
`pip3 install -r requirements.txt`	Install from requirements file
`python3 -m venv venv`	Create a virtual environment
`source venv/bin/activate`	Activate the virtual environment
`pip3 list`	List installed packages
`python3 -m http.server 8080`	Spin up a quick HTTP file server on port 8080

09 Authorized Lab Tools — Syntax Reference

WARNING — AUTHORIZED USE ONLY. Install and use these tools exclusively on networks, systems, and lab environments you own or have explicit written permission to test. Unauthorized scanning or exploitation is illegal.

```
sudo apt update

sudo apt install nmap tcpdump tshark nikto gobuster sqlmap -y
```

nmap — Network Mapper

nmap is the industry-standard network discovery and port scanning tool. Use it during authorized assessments to inventory hosts, open ports, running services, and OS fingerprints.

Command	Description
`nmap 192.168.1.1`	Basic scan of a single host
`nmap 192.168.1.0/24`	Scan entire /24 subnet
`nmap -sV 192.168.1.1`	Service version detection
`nmap -O 192.168.1.1`	OS fingerprinting (may need sudo)
`nmap -A 192.168.1.1`	Aggressive scan — OS, version, scripts, traceroute
`nmap -p 22,80,443 192.168.1.1`	Scan specific ports only
`nmap -p- 192.168.1.1`	Scan all 65,535 ports
`nmap -sU -p 53,161 192.168.1.1`	UDP scan on ports 53 and 161
`nmap -sn 192.168.1.0/24`	Ping sweep — discover live hosts, no port scan
`nmap --script vuln 192.168.1.1`	Run vulnerability NSE scripts
`nmap -oN output.txt 192.168.1.1`	Save output in normal text format
`nmap -oX output.xml 192.168.1.1`	Save output in XML for programmatic parsing
`nmap -T4 192.168.1.1`	Faster timing — T0 slowest through T5 fastest
`nmap -sT 192.168.1.1`	Connect scan — use when raw sockets unavailable

TIP — On Android with PRoot, nmap's SYN scan (-sS) needs raw sockets and may fail. Add -sT to fall back to a connect scan, which works reliably in PRoot.

tcpdump — Packet Capture

tcpdump captures and analyzes network traffic in real time from the command line. Use it to verify traffic flow, investigate anomalies, and capture evidence on authorized networks. Always save captures with -w for later analysis.

Command	Description
`tcpdump -i eth0`	Capture on interface eth0
`tcpdump -i any`	Capture on all interfaces
`tcpdump -n -i eth0`	Disable DNS resolution — faster, cleaner output
`tcpdump -nn -i eth0`	Disable DNS and port name resolution
`tcpdump port 80 -i eth0`	Capture only HTTP traffic
`tcpdump host 192.168.1.10 -i eth0`	Filter to/from a specific host
`tcpdump -w capture.pcap -i eth0`	Write capture to file for later analysis
`tcpdump -r capture.pcap`	Read and display a saved pcap
`tcpdump -c 100 -i eth0`	Capture exactly 100 packets then stop
`tcpdump 'tcp[tcpflags] & tcp-syn != 0'`	Capture only TCP SYN packets

TIP — Remove or encrypt captures after the engagement. Open .pcap files in Wireshark on a workstation for deep packet inspection.

tshark — Terminal Wireshark

tshark is the command-line Wireshark interface. It reads pcap files and applies Wireshark display filter syntax.

Command	Description
`tshark -r capture.pcap`	Read and display a saved pcap
`tshark -r capture.pcap -Y 'http'`	Filter HTTP traffic only
`tshark -r capture.pcap -Y 'dns'`	Filter DNS traffic only
`tshark -r capture.pcap -T fields -e ip.src -e ip.dst`	Extract source and destination IPs
`tshark -i eth0 -w out.pcap`	Live capture to file

nikto — Web Server Scanner

nikto scans web servers for known vulnerabilities, misconfigurations, and exposed files. Authorized web application assessments only.

Command	Description
`nikto -h http://192.168.1.10`	Basic scan against a web server
`nikto -h https://192.168.1.10 -ssl`	Scan an HTTPS target
`nikto -h 192.168.1.10 -p 8080`	Scan on a non-standard port
`nikto -h 192.168.1.10 -o report.html -Format htm`	Save results as HTML report
`nikto -h 192.168.1.10 -Tuning 9`	Run SQL injection checks

gobuster — Directory and DNS Brute-Forcer

gobuster enumerates hidden web directories, files, and subdomains using wordlists. Used during authorized web application assessments.

Command	Description
`gobuster dir -u http://target -w /usr/share/wordlists/dirb/common.txt`	Directory brute-force with common wordlist
`gobuster dir -u http://target -w wordlist.txt -x php,html`	Enumerate .php and .html files
`gobuster dns -d example.com -w wordlist.txt`	Subdomain enumeration
`gobuster dir -u http://target -w wordlist.txt -t 50`	Run with 50 threads
`gobuster dir -u http://target -w wordlist.txt -o results.txt`	Save results to file

sqlmap — SQL Injection Testing

sqlmap automates detection and exploitation of SQL injection vulnerabilities. Authorized penetration testing only.

WARNING — Never run sqlmap against a target without explicit written authorization. sqlmap can alter, dump, or damage the target database.

Command	Description
`sqlmap -u 'http://target/page?id=1'`	Basic injection test on a GET parameter
`sqlmap -u 'http://target/page?id=1' --dbs`	List all databases if vulnerable
`sqlmap -u 'http://target/page?id=1' -D dbname --tables`	List tables in a specific database
`sqlmap -u 'http://target/page?id=1' --data='user=x&pass;=y'`	Test a POST parameter
`sqlmap -u 'http://target' --forms`	Auto-detect and test all forms
`sqlmap -u 'http://target/page?id=1' --level=5 --risk=3`	Maximum detection depth
`sqlmap -u 'http://target/page?id=1' --batch`	Non-interactive mode (auto-answer prompts)

10 Status Script and Automation

Create an operator status script to confirm environment health at the start of each session:

```
nano ~/greynoc-edc/scripts/status.sh
```

Paste the following content:

```
#!/usr/bin/env bash

echo '=== GreyNOC EDC Status ==='

date

echo

echo "User: $(whoami)"

echo "Host: $(hostname)"

echo

echo 'Python:' && python3 --version

echo 'SSH:' && ssh -V

echo

echo 'Network:'

ip addr 2>/dev/null | head -40
```

```
chmod +x ~/greynoc-edc/scripts/status.sh

~/greynoc-edc/scripts/status.sh
```

TIP — Add the status script to your shell profile for automatic execution on login: echo '~/greynoc-edc/scripts/status.sh' >> ~/.bashrc

Capture Output to Log File

Append any tool output to a dated log file with tee:

```
nmap -sV 192.168.1.1 | tee ~/greynoc-edc/logs/nmap-$(date +%Y%m%d).txt
```

11 Daily Operator Reference

Action	Command
Start Linux workspace	Open Termux → linux-phone
Exit back to Termux	exit
Update Debian	sudo apt update && sudo apt upgrade -y
Install a package	sudo apt install -y
Start a tmux session	tmux new -s ops
Reattach to a session	tmux attach -t ops
Check Python version	python3 --version
Check SSH version	ssh -V
Check IP addresses	ip a
Run status check	~/greynoc-edc/scripts/status.sh
Quick DNS lookup	dig +short example.com A
Quick port check	nc -zv 192.168.1.1 22
SSH to authorized host	ssh user@192.168.1.1

Quick Reference Session

```
linux-phone

tmux new -s ops

sudo apt update && sudo apt upgrade -y

~/greynoc-edc/scripts/status.sh

# ... do your authorized work ...

exit # exit tmux shell

exit # exit Debian back to Termux
```

12 Field Notes and Evidence Hygiene

Operators should document all actions with professional, factual, permission-aware notes. Good notes protect you, your clients, and the integrity of findings.

Recommended Note Format

```
Date:

Operator:

Authorized environment:

Purpose:

Commands or actions performed:

Observed result:

Follow-up needed:

Sensitive data captured? yes/no

Sanitized copy created? yes/no
```

Evidence Handling Rules

Rule	Guidance
Work on a copy	Never edit original evidence — always work on a sanitized duplicate
Sanitize before sharing	Remove IPs, hostnames, credentials, customer names, paths
Verify the copy	Review the sanitized copy line by line before publishing
No plain-text secrets	Do not store credentials, keys, or tokens in plain text on the phone
Metadata stripping	Check file metadata (exiftool on images) before external release

13 OpSec and Public Posting Rules

Before posting screenshots, videos, guides, or operator notes, check for each of the following categories of sensitive data:

Category	Examples to Redact
Network identifiers	IP addresses, hostnames, MAC addresses, Wi-Fi SSIDs
Credentials and keys	Passwords, API keys, private keys, tokens, hashes
Account identifiers	Usernames, email addresses, employee IDs
Client and business data	Customer names, ticket numbers, contract details
Location data	GPS coordinates, badge IDs, license plates, schedules
Personal identifiers	Faces, reflections in screens, voices in recordings
File system clues	Terminal paths, folder names, internal hostnames in prompts
Document metadata	Author fields, revision history, embedded coordinates
Browser context	Open tabs visible in screenshots, browser history in address bar

WARNING — Always work on a copy. Sanitize the copy. Verify the copy. Publish only the verified public-safe copy.

14 Troubleshooting Reference

Problem	Recommended Action
pkg update fails	Check Wi-Fi, rerun pkg update, then try pkg upgrade -y again
proot-distro install fails	Check storage and network stability; remove with proot-distro remove debian then reinstall
sudo does not work inside Debian	Confirm you created a normal user and ran usermod -aG sudo greyuser
Permission denied on a command	Confirm whether the command belongs in Termux or Debian; use sudo only inside Debian after user setup
Command not found	Install the missing package with apt install or confirm you are in the right shell environment
Phone gets very hot	Pause package installs, plug in the device, close background apps, avoid long compile jobs
Storage fills up	Run: sudo apt autoremove -y && sudo apt clean then move old lab files off-device
Tool works on laptop but not phone	Android, PRoot, kernel, or hardware limits may block the feature — raw sockets, monitor mode, etc.
nmap scan is very slow or fails	Add -T4 for faster timing; add -sT if raw socket scan fails in PRoot environment
tmux session was lost	Run: tmux attach to recover; always start sessions inside tmux to prevent this

Helpful Diagnostic Commands

```
proot-distro list # list installed distributions

proot-distro remove debian # remove a broken Debian install

sudo apt update # refresh package index

sudo apt autoremove -y # remove unused packages

sudo apt clean # clear package cache

df -h # check available disk space

free -h # check memory usage

ip a # show network interfaces and addresses
```

15 Operator Safety Rules

01	Get written permission before touching any network or system you do not own.
02	Keep client, employer, and production data off public posts and personal devices.
03	Use synthetic examples in screenshots and public demonstrations — never real targets.
04	Do not store secrets, keys, or credentials in plain text anywhere on the phone.
05	Keep Android, Termux, and Debian packages updated at the start of each session.
06	Use screen lock and full-device encryption — Android Settings > Security.
07	Separate personal, lab, and client materials into distinct folders at all times.
08	Record what you did, when, and why in clean, factual field notes.
09	Delay or avoid social media posts from live work, travel, incidents, or sensitive sites.
10	When unsure whether an action is authorized — stop and ask before proceeding.

A Legal and Safety Disclaimer

This document is published by GreyNOC (greynoc.com) for public education, defensive operations, and authorized system administration only. GreyNOC does not endorse, support, or provide guidance for unauthorized access, scanning, exploitation, credential attacks, interception, evasion, or bypassing access controls on systems, networks, or accounts without explicit written permission.

Operators are solely responsible for ensuring compliance with applicable federal law (Computer Fraud and Abuse Act, 18 U.S.C. § 1030), state law (Michigan MCL 752.791), organizational policy, service terms, and written authorization from system and network owners before conducting any active testing, scanning, or data collection activity.

This guide is provided as-is under the MIT License. GreyNOC assumes no liability for misuse of the information or tools described herein.

References

Resource	URL / Source
F-Droid official site	f-droid.org
Termux Wiki	wiki.termux.com
PRoot Distro GitHub	github.com/termux/proot-distro
Debian package documentation	packages.debian.org
nmap reference guide	nmap.org/book/man.html
tmux cheat sheet	tmuxcheatsheet.com
OWASP Testing Guide	owasp.org/www-project-web-security-testing-guide
GreyNOC public guidance	greynoc.com
CFAA overview	justice.gov/criminal/cybercrime